

Genetic Algorithms

In this lab, you will implement and analyze a simple *Genetic Algorithm (GA)*. The aim is to understand how a GA evolves a population of candidate solutions using *fitness evaluation, selection, crossover, and mutation*.

The lab starts with a simple binary optimization problem and then extends the same idea to a machine learning feature-selection task. You should focus not only on getting the code to run, but also on understanding how design choices such as population size, mutation rate, crossover rate, tournament size, and fitness function affect the search process.

Learning Objectives

By the end of this lab, you should be able to implement a basic Genetic Algorithm from scratch, represent candidate solutions as chromosomes, calculate fitness values, apply selection, crossover, and mutation, interpret convergence plots, and explain how GAs can be applied to feature selection and optimization problems in Data Engineering and AI.

Use Python 3 with the following libraries:

- Numpy
- Matplotlib
- scikit-learn
- pandas

If needed, install packages using:

```
pip install numpy matplotlib scikit-learn pandas
```

Activity 1: Optimizing a Binary String

In this activity, you will implement a GA to solve the “maximise ones” problem. The goal is to find a binary chromosome with as many 1s as possible. Each chromosome is a binary string such as:

1 0 1 1 0 0 1 0 1 1

The fitness function is simply:

fitness = number of 1s

This task is intentionally simple so that you can focus on the mechanics of the Genetic Algorithm.

Activity 2: Implement Selection, Crossover, and Mutation

You will implement three main operators. *Tournament selection* randomly selects a small group of individuals and chooses the one with the highest fitness. *One-point crossover* combines two parent chromosomes by swapping sections after a randomly selected crossover point. *Bit-flip mutation* randomly changes a bit from 0 to 1 or from 1 to 0.

You should **manually test** these operators using two parent chromosomes before running the full GA.

Activity 3: Build the Full Genetic Algorithm

You will combine population initialization, fitness evaluation, selection, crossover, mutation, and population replacement into a complete GA. You will then plot the best and mean fitness values across generations. A typical convergence plot should show whether the population is improving over time. If the best fitness improves but the mean fitness remains low, the population may still be diverse. If the mean fitness becomes very close to the best fitness, the population may have converged.

Activity 4: Experiment with GA Parameters

Run the GA several times using different parameter settings. Compare the default setting with low mutation, high mutation, small population, large population, and low crossover rate.

Record your observations in a table:

Experiment	Best fitness	Behaviour observed
Default settings		
Low mutation		
High mutation		
Small population		
Large population		
Low crossover		

Consider the following questions:

- What happens when the mutation rate is too low?
- What happens when mutation is too high?
- How does population size affect convergence?
- Why can results differ between runs?

Activity 5: Feature Selection with a Genetic Algorithm

In this activity, the chromosome represents selected features for a machine learning model. A value of 1 means the feature is selected, while 0 means the feature is excluded. The lab uses the breast cancer dataset from scikit-learn. A Decision Tree classifier is evaluated using cross-validation. The fitness function combines validation accuracy with a small penalty for selecting too many features:

$$\text{fitness} = \text{cross-validation accuracy} - \text{feature penalty}$$

This encourages the GA to find feature subsets that are both accurate and compact.

Activity 6: Reflection and Discussion

Answer the following questions:

1. Why is a binary chromosome suitable for feature selection?

2. Why should validation or cross-validation accuracy be used instead of training accuracy?
3. What role does the penalty term play in the feature-selection fitness function?
4. How might the GA behave differently if the dataset had 5,000 features?
5. What are the risks of using a GA for machine learning model selection?
6. How could this GA be improved for larger data engineering problems?

Lab Deliverables

Submit a short lab report and python code.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier

np.random.seed(42)

# Load breast cancer dataset
breast_cancer_data = load_breast_cancer()
X = breast_cancer_data.data
y = breast_cancer_data.target
feature_names = breast_cancer_data.feature_names

#####
# Define a function to compute fitness for the maximise-ones problem
#####
def ones_fitness(chromosome):
    """Calculate fitness for the maximise-ones problem."""
    return np.sum(chromosome)

#####
# Define a function to create a random binary chromosome
#####
def create_population(pop_size, chromosome_length):
    """Create a population of random binary chromosomes."""
    population = []

    for i in range(pop_size):
        """Create one random binary chromosome."""
        chromosome = np.random.randint(0, 2, chromosome_length)
        population.append(chromosome)

    return np.array(population)

#####
# Define a function for tournament selection
#####
def tournament_selection(population, fitness_values, tournament_size):
    """Select one parent using tournament selection."""
    selected_indices = np.random.choice(len(population), tournament_size,
replace=False)

    selected_fitness = fitness_values[selected_indices]

```

```

best_position = np.argmax(selected_fitness)
best_index = selected_indices[best_position]

return population[best_index].copy()

#####
# Define a function for one-point crossover
#####
def one_point_crossover(parent1, parent2, crossover_rate):
    """Create two children using one-point crossover."""
    random_value = np.random.rand()

    if random_value < crossover_rate:
        point = np.random.randint(1, len(parent1))

        child1_left = parent1[:point]
        child1_right = parent2[point:]

        child2_left = parent2[:point]
        child2_right = parent1[point:]

        child1 = np.concatenate((child1_left, child1_right))
        child2 = np.concatenate((child2_left, child2_right))

        return child1, child2

    return parent1.copy(), parent2.copy()

#####
# Define a function for mutation
#####
def mutate(chromosome, mutation_rate):
    """Apply bit-flip mutation to a chromosome."""
    mutated_chromosome = chromosome.copy()

    for i in range(len(mutated_chromosome)):
        random_value = np.random.rand()

        if random_value < mutation_rate:
            mutated_chromosome[i] = 1 - mutated_chromosome[i]

    return mutated_chromosome

#####
# Define Genetic Algorithm Functions

```

```
#####
def genetic_algorithm( fitness_function, chromosome_length, pop_size,
generations, crossover_rate, mutation_rate, tournament_size):
    """Run a binary Genetic Algorithm."""

    population = create_population(pop_size, chromosome_length)

    best_fitness_history = []
    mean_fitness_history = []

    for generation in range(generations):

        fitness_values = []

        for chromosome in population:
            score = fitness_function(chromosome)
            fitness_values.append(score)

        fitness_values = np.array(fitness_values)

        best_fitness = np.max(fitness_values)
        mean_fitness = np.mean(fitness_values)

        best_fitness_history.append(best_fitness)
        mean_fitness_history.append(mean_fitness)

        new_population = []

        while len(new_population) < pop_size:

            parent1 = tournament_selection( population, fitness_values,
tournament_size )

            parent2 = tournament_selection( population, fitness_values,
tournament_size )

            child1, child2 = one_point_crossover( parent1, parent2,
crossover_rate )

            child1 = mutate(child1, mutation_rate)
            child2 = mutate(child2, mutation_rate)

            new_population.append(child1)

        if len(new_population) < pop_size:
```

```

        new_population.append(child2)

    population = np.array(new_population)

    # Evaluate final population
    final_fitness_values = []

    for chromosome in population:
        score = fitness_function(chromosome)
        final_fitness_values.append(score)

    final_fitness_values = np.array(final_fitness_values)

    best_index = np.argmax(final_fitness_values)

    best_solution = population[best_index]
    best_score = final_fitness_values[best_index]

    return ( best_solution, best_score, best_fitness_history,
mean_fitness_history)

def plot_convergence(best_history, mean_history, title):
    """Plot best and mean fitness across generations."""

    plt.figure(figsize=(9, 5))

    plt.plot(best_history, label="Best fitness")
    plt.plot(mean_history, label="Mean fitness")

    plt.xlabel("Generation")
    plt.ylabel("Fitness")
    plt.title(title)

    plt.legend()
    plt.grid(True)

    plt.show()

#####
# Task 1: Maximise Ones Problem
#####
def run_maximise_ones_demo():
    """Run the maximise-ones Genetic Algorithm example."""

```

```

    best_solution, best_score, best_history, mean_history =
genetic_algorithm(
    fitness_function=ones_fitness,
    chromosome_length=20,
    pop_size=30,
    generations=50,
    crossover_rate=0.8,
    mutation_rate=0.01,
    tournament_size=3
)

print("Maximise Ones Problem")
print("Best solution:", best_solution)
print("Best fitness:", best_score)

plot_convergence( best_history, mean_history, "GA Convergence: Maximise
Ones" )

#####
# Task 2: Maximise Ones Problem
#####
def run_parameter_experiments():
    """Run Genetic Algorithm experiments with different parameters."""

    experiments = [
        ["Default settings", 30, 50, 0.8, 0.01, 3],
        ["Low mutation", 30, 50, 0.8, 0.001, 3],
        ["High mutation", 30, 50, 0.8, 0.2, 3],
        ["Small population", 10, 50, 0.8, 0.01, 3],
        ["Large population", 100, 50, 0.8, 0.01, 3],
        ["Low crossover", 30, 50, 0.2, 0.01, 3]
    ]

    plt.figure(figsize=(10, 6))

    print("Parameter Experiment Results")

    for experiment in experiments:

        name = experiment[0]
        pop_size = experiment[1]
        generations = experiment[2]
        crossover_rate = experiment[3]

```



```

        mutation_rate = experiment[4]
        tournament_size = experiment[5]

        best_solution, best_score, best_history, mean_history =
genetic_algorithm(
            fitness_function=ones_fitness,
            chromosome_length=20,
            pop_size=pop_size,
            generations=generations,
            crossover_rate=crossover_rate,
            mutation_rate=mutation_rate,
            tournament_size=tournament_size
        )

        print( name, "Best fitness:", best_score, "Best solution:",
best_solution)

        plt.plot(best_history, label=name)

        plt.xlabel("Generation")
        plt.ylabel("Best fitness")
        plt.title("Parameter Experiment Comparison")

        plt.legend()
        plt.grid(True)

        plt.show()

#####
# Define a function to compute fitness for the feature selection problem
#####

def feature_selection_fitness(chromosome):
    """
    Calculate fitness for feature selection using
    cross-validation accuracy and a feature penalty.
    """

    number_selected = np.sum(chromosome)

    # Avoid selecting zero features
    if number_selected == 0:
        return 0

```

```

    selected_features = np.where(chromosome == 1)[0]

    X_selected = X[:, selected_features]

    model = DecisionTreeClassifier(random_state=42)

    scores = cross_val_score( model, X_selected, y, cv=5, scoring="accuracy" )

    accuracy = np.mean(scores)

    penalty = 0.01 * ( number_selected / len(chromosome) )

    fitness = accuracy - penalty

    return fitness

#####
# Task 3: Feature Selection Problem
#####

def run_feature_selection_demo():
    """Run the Genetic Algorithm for feature selection."""

    number_of_features = X.shape[1]

    best_solution, best_score, best_history, mean_history = genetic_algorithm(
        fitness_function=feature_selection_fitness,
        chromosome_length=number_of_features,
        pop_size=30,
        generations=30,
        crossover_rate=0.8,
        mutation_rate=1 / number_of_features,
        tournament_size=3
    )

    selected_indices = np.where(best_solution == 1)[0]
    selected_names = feature_names[selected_indices]

    print("Feature Selection Problem")
    print("Number of available features:", number_of_features)

    print("Best chromosome:", best_solution)

    print( "Number of selected features:", len(selected_indices) )

```

```

print( "Selected feature indices:", selected_indices )

print("Selected feature names:")

for name in selected_names:
    print(name)

print("Best GA fitness:", best_score)

# Accuracy using all features
model = DecisionTreeClassifier(random_state=42)

all_feature_scores = cross_val_score( model, X, y, cv=5,
scoring="accuracy" )

all_feature_accuracy = np.mean(all_feature_scores)

print( "Accuracy using all features:", all_feature_accuracy)

plot_convergence( best_history, mean_history, "GA Convergence: Feature
Selection" )

def lab_05():
    """Run all lab demonstrations."""

    print("ENGE707 Week 5 Lab: Genetic Algorithms")

    print("Running Maximise Ones Demo...")
    run_maximise_ones_demo()
    print("Running Parameter Experiments...")
    run_parameter_experiments()
    print("Running Feature Selection Demo...")
    run_feature_selection_demo()

    print("Lab complete")

if __name__ == "__main__":
    lab_05()

```